

REMOTE SENSING · OPERATIONS

# 遥感算法容器封装 与本机部署操作手册

Docker / Kubernetes / GPU / 离线交付

适用算法镜像：0.3.0-kmeans1

文档版次：1.1

编制日期：2026 年 9 月 8 日

本手册规定遥感算法封装测试版的部署、配置、执行、停止、验证与交付操作。各操作条目包含输入填写要求、指令及参数、终端输出和完成判定，适用于当前本机测试环境及已准备基础依赖的 CPU 离线运行环境。

适用基线：算法镜像 0.3.0-kmeans1、源码提交 e59ce62。本文引用的实测输出来自 2026-09-08 已保存记录，动态字段不作为固定验收值。

# 目录

|                           |           |
|---------------------------|-----------|
| <b>第一章 文档约定与操作流程</b>      | <b>1</b>  |
| 1.1 适用范围与版本               | 1         |
| 1.2 指令与输出表示规则             | 1         |
| 1.3 标准操作顺序                | 1         |
| <b>第二章 环境初始化与状态确认</b>     | <b>2</b>  |
| 2.1 ENV-01 设置操作终端         | 2         |
| 2.2 ENV-02 检查磁盘和 Docker   | 3         |
| 2.3 当前存储映射                | 4         |
| <b>第三章 任务配置文件规范</b>       | <b>5</b>  |
| 3.1 CFG-01 建立本次任务 env     | 5         |
| 3.2 需由使用者填写的字段            | 6         |
| 3.3 环境与资源字段               | 6         |
| 3.4 配置文件的填写样式             | 6         |
| <b>第四章 算法执行与参数参考</b>      | <b>8</b>  |
| 4.1 RUN-01 生成 KMEANS 验收样本 | 8         |
| 4.2 RUN-02 执行 env 配置任务    | 9         |
| 4.3 RUN-03 检查结果和完成标记      | 10        |
| 4.4 KMEANS 参数表            | 11        |
| 4.5 RUN-04 执行 MNDWI 示例    | 11        |
| 4.6 MNDWI 参数表             | 12        |
| 4.7 算法命令行接口               | 13        |
| <b>第五章 运行监控、停止和重跑</b>     | <b>14</b> |
| 5.1 CTL-01 查询状态与日志        | 14        |
| 5.2 CTL-02 停止指定容器         | 15        |
| 5.3 重跑规则                  | 15        |

|                                      |           |
|--------------------------------------|-----------|
| 5.4 退出码参考 . . . . .                  | 16        |
| <b>第六章 CPU 与 Docker GPU 验收</b>       | <b>17</b> |
| 6.1 TEST-01 CPU 封装回归 . . . . .       | 17        |
| 6.2 TEST-02 Docker GPU 计算 . . . . .  | 18        |
| <b>第七章 Kubernetes 操作与 GPU 调度</b>     | <b>20</b> |
| 7.1 K8S-01 检查节点与导入算法镜像 . . . . .     | 20        |
| 7.2 K8S-02 CPU Job 回归 . . . . .      | 21        |
| 7.3 K8S-03 GPU 完整调度测试 . . . . .      | 21        |
| 7.4 K8S-04 手动提交 GPU Job . . . . .    | 23        |
| 7.5 手动 GPU Job 模板参数 . . . . .        | 24        |
| 7.6 K8S-05 查看、暂停与保留 Job 记录 . . . . . | 24        |
| <b>第八章 本地集群维护</b>                    | <b>26</b> |
| 8.1 MAINT-01 创建缺失的 kind 集群 . . . . . | 26        |
| 8.2 MAINT-02 复装本机 GPU 接入 . . . . .   | 26        |
| 8.3 组件与路径 . . . . .                  | 27        |
| <b>第九章 构建、安全扫描与离线交付</b>              | <b>28</b> |
| 9.1 REL-01 构建新的镜像版本 . . . . .        | 28        |
| 9.2 REL-02 扫描镜像漏洞 . . . . .          | 29        |
| 9.3 已记录安全状态 . . . . .                | 30        |
| 9.4 REL-03 导出单个镜像 . . . . .          | 30        |
| 9.5 REL-04 加载已有 CPU 离线交付包 . . . . .  | 30        |
| 9.6 REL-05 使用现有打包入口 . . . . .        | 32        |
| <b>第十章 源码推送与文件发布</b>                 | <b>33</b> |
| 10.1 GIT-01 推送已提交开发分支 . . . . .      | 33        |
| <b>第十一章 故障定位与交付验收</b>                | <b>34</b> |
| 11.1 故障对照表 . . . . .                 | 34        |
| 11.2 DIAG-01 保存集群故障记录 . . . . .      | 34        |
| 11.3 交付完成条件 . . . . .                | 35        |
| 11.4 待真实代码到达后填写的信息 . . . . .         | 35        |

|                           |           |
|---------------------------|-----------|
| <b>第十二章 附录：脚本、证据与文档维护</b> | <b>37</b> |
| 12.1 脚本与实际作用 . . . . .    | 37        |
| 12.2 日志和结果字段 . . . . .    | 37        |
| 12.3 实测证据位置 . . . . .     | 37        |
| 12.4 文档源文件与编译 . . . . .   | 38        |

# 第一章 文档约定与操作流程

## 1.1 适用范围与版本

本手册适用于 Linux amd64 上的遥感算法容器封装测试版。已接入算法为 MNDWI、KMEANS；smoke 用于文件流转诊断。GPU 章节验证设备接入与 CUDA 运算，不改变上述 CPU 示例的计算后端。当前 Kubernetes 实现为本机单节点 kind 集群 rs-lab。

环境和测试基线来自同日保存的部署记录；本次文档修订不安装环境、不运行运维命令、不合并业务脚本。正式目标平台、真实算法、网络与安全门禁须按实际交付要求验收。

## 1.2 指令与输出表示规则

| 表示方式       | 含义                            |
|------------|-------------------------------|
| 执行命令       | 复制到 Bash 终端执行；不包含 \$ 提示符      |
| 配置文件内容     | 保存到指定 env 文件；不是直接执行的 shell 命令 |
| 需填写内容      | 使用前应替换或核对的变量与路径               |
| 终端输出（实测记录） | 从已保存证据中摘录；保留来源路径              |
| 终端输出（格式示意） | 按现有脚本行为构造的显示示例；不宣称重新实测        |
| 无输出        | 命令成功时可能不打印文字；应查看退出码或后续状态      |

同一操作条目的变量在当前终端有效。跨终端操作必须重新填写容器名或 Job 名。代码块中的反斜杠是 Bash 续行符；复制命令推荐使用配套 Markdown 文本版，避免 PDF 的视觉折行进入命令。

## 1.3 标准操作顺序

| 阶段    | 操作                         | 形成的记录           |
|-------|----------------------------|-----------------|
| 准备    | 确认项目路径、数据盘、镜像及访问权限         | 环境检查输出          |
| 配置    | 建立独立任务 env，填写输入、输出和算法参数    | env 配置文件        |
| 执行    | 启动、保存 stdout/stderr、记录退出码  | 日志与退出码文件        |
| 判定    | 检查 result 文件与 success.json | 结果及完成标记         |
| 停止或恢复 | 停止指定任务；失败时采用新目录重跑          | 中止记录或新任务结果      |
| 验收与交付 | 执行相应测试、扫描、导出和校验            | 验收报告、镜像及 SHA256 |

## 第二章 环境初始化与状态确认

### 2.1 ENV-01 设置操作终端

#### 用途

设定项目目录、算法镜像和 Kubernetes 工具路径。

#### 执行前提

本机目录已存在；使用有 Docker 权限的普通用户，日常运行不使用 sudo。

#### 需填写内容

PROJECT\_ROOT：项目根目录；IMAGE：准备运行的已加载镜像。本机示例可直接使用下列值。

#### 执行命令

```
export PROJECT_ROOT=/mnt/robot_disk/remote_sensing_project
export IMAGE=remote-sensing-framework:0.3.0-kmeans1
export PATH=/mnt/robot_disk/k8s/bin:$PATH
export KUBECONFIG=/mnt/robot_disk/k8s/config/kubeconfig
cd "$PROJECT_ROOT"
printf 'PROJECT_ROOT=%s\nIMAGE=%s\n' "$PWD" "$IMAGE"
```

| 指令 / 变量             | 说明                       |
|---------------------|--------------------------|
| export              | 将变量传递给后续子进程；重新开终端后需再次设置  |
| cd "\$PROJECT_ROOT" | 切换工作目录；引号支持路径中的空格        |
| PATH                | 追加本机 kind 和 kubectl 所在目录 |
| KUBECONFIG          | 指定集群访问配置；不得作为公开交付附件      |
| IMAGE               | 选择算法镜像；不触发拉取、构建或 GPU 启用  |

#### 终端输出（格式示意）

```
PROJECT_ROOT=/mnt/robot_disk/remote_sensing_project
IMAGE=remote-sensing-framework:0.3.0-kmeans1
```

示意中的任务名、时间、目录、哈希和耗时以实际运行值为准；该输出用于说明格式，不构成本次重新运行证据。

#### 完成判定与异常处理

打印的项目路径和镜像名与计划一致。cd 失败时先修正 PROJECT\_ROOT；不要在其他工作目录继续执行相对路径命令。

## 2.2 ENV-02 检查磁盘和 Docker

### 用途

确认数据盘挂载、Docker 服务、存储目录和镜像可用。

### 执行前提

已完成 ENV-01；本机 GPU 和 Kubernetes 检查不作为 CPU 运行的必需前提。

### 需填写内容

无额外必填项；目标机器应将数据盘路径替换为已确认的实际路径。

### 执行命令

```
findmnt -T "$PROJECT_ROOT"
df -h /mnt/robot_disk
systemctl is-active docker
docker info --format 'DockerRootDir={{.DockerRootDir}}'
docker image inspect "$IMAGE" --format '{{.Id}}'
bash scripts/check-environment.sh .
```

| 参数                     | 说明                           |
|------------------------|------------------------------|
| findmnt -T 路径          | 显示承载该路径的文件系统                 |
| df -h                  | 按便于阅读的单位显示容量和剩余空间            |
| --format               | 提取指定字段，避免输出整份配置              |
| check-environment.sh . | 检查工具与 CPU 基础条件；句点用于显示当前目录所在盘 |

### 终端输出（格式示意）

```
active
DockerRootDir=/mnt/robot_disk/docker-engine
sha256:e9bcee28fc600022a4a919897528e703e1c30d14730b32679c57a4c9a7ea3c89
CPU_PREREQUISITE_EXIT=0
```

示意中的任务名、时间、目录、哈希和耗时以实际运行值为准；该输出用于说明格式，不构成本次重新运行证据。

### 完成判定与异常处理

Docker 为 active，CPU 基础检查为 0，镜像 inspect 成功。本机 DockerRootDir 应落在 /mnt/robot\_disk 下。GPU\_CHECK=NOT\_READY 不代表 CPU 不可运行，但应停止进入 GPU 验收流程。

## 2.3 当前存储映射

| 用户填写的宿主目录     | 容器内固定目录           | 读写用途               |
|---------------|-------------------|--------------------|
| INPUT_DIR     | /data/input       | 只读输入               |
| OUTPUT_DIR    | /data/output      | 持久输出；新任务使用新目录      |
| WORK_DIR      | /data/work        | 大型临时文件预留，现有两个示例未使用 |
| 项目 data/input | kind 内 /rs-input  | Pod 输入来源           |
| 项目 outputs    | kind 内 /rs-output | Pod 结果来源           |

本机数据盘为 /dev/sdb2, ext4 挂载于 /mnt/robot\_disk; 记录时可用约 858 GiB。Docker 数据目录为 /mnt/robot\_disk/docker-engine, 宿主 containerd 数据目录为 /mnt/robot\_disk/containerd。程序和部分系统服务文件仍位于系统分区。



## 第三章 任务配置文件规范

### 3.1 CFG-01 建立本次任务 env

#### 用途

将运行配置保存为独立文件，避免不同任务相互覆盖。

#### 执行前提

执行 ENV-01；任务输出目录尚未包含已完成结果。

#### 需填写内容

TASK\_ID：本次任务标识。示例自动生成；正式使用可采用工单号加时间。生成文件后须核对 INPUT\_DIR、RS\_INPUT 和 RS\_PARAMS。

#### 执行命令

```
TASK_ID="kmeans-$(date +%Y%m%d-%H%M%S)-$$"
ENV_FILE="$PROJECT_ROOT/.env.$TASK_ID"
cat > "$ENV_FILE" <<EOF
IMAGE=$IMAGE
INPUT_DIR=./data/input/kmeans-example
OUTPUT_DIR=./outputs/$TASK_ID
WORK_DIR=./work/$TASK_ID
CONTAINER_NAME=rs-$TASK_ID
CPU_LIMIT=1
MEMORY_LIMIT=1g
STOP_TIMEOUT=10
RS_ALGORITHM=KMEANS
RS_INPUT=/data/input/kmeans-input.tif
RS_PARAMS={"clusters":3,"seed":0,"max_iter":100,"max_pixels":1000000}
RS_TASK_ID=$TASK_ID
EOF
printf 'ENV_FILE=%s\nTASK_ID=%s\n' "$ENV_FILE" "$TASK_ID"
```

#### 终端输出（格式示意）

```
ENV_FILE=/mnt/robot_disk/remote_sensing_project/.env.kmeans-20260908-140000-12345
TASK_ID=kmeans-20260908-140000-12345
```

示意中的任务名、时间、目录、哈希和耗时以实际运行值为准；该输出用于说明格式，不构成本次重新运行证据。

完成判定与异常处理

env 文件存在, INPUT\_DIR 指向真实输入目录, RS\_INPUT 是容器内输入文件路径。模板中的 EOF 用于在创建时展开受控变量, env-run.py 实际读取文件时不执行 shell。

3.2 需由使用者填写的字段

| 字段             | 填写规则              | 示例                           |
|----------------|-------------------|------------------------------|
| INPUT_DIR      | 必填; 宿主目录          | ./data/input/kmeans-example  |
| OUTPUT_DIR     | 必填; 每次新的宿主输出目录    | ./outputs/kmeans-001         |
| RS_ALGORITHM   | 选定已接入算法, 区分大小写    | KMEANS                       |
| RS_INPUT       | 必填; 容器内文件名        | /data/input/kmeans-input.tif |
| RS_PARAMS      | JSON 对象; 内容见算法参数表 | {"clusters":3,"seed":0}      |
| CONTAINER_NAME | 建议填写; 当前任务唯一容器名   | rs-kmeans-001                |
| RS_TASK_ID     | 建议填写; 日志关联标识      | kmeans-001                   |

3.3 环境与资源字段

| 字段           | 模板值 / 缺省行为                   | 设置影响         |
|--------------|------------------------------|--------------|
| IMAGE        | 模板为 0.3.0-kmeans1; 运行脚本要求提供  | 选择已加载镜像      |
| WORK_DIR     | 模板独立目录; 未设置则 OUTPUT_DIR/work | 持久工作空间       |
| CPU_LIMIT    | 1                            | CPU 配额上限     |
| MEMORY_LIMIT | 模板 1g; run.sh 缺省 512m        | 普通内存限制, 不是显存 |
| STOP_TIMEOUT | 10; 非负整数                     | 正常终止宽限秒数     |

INPUT\_DIR、OUTPUT\_DIR、WORK\_DIR 的相对路径以 env 文件所在目录为基准。文件支持中文和空格路径, 不支持变量展开、命令替换及行尾注释。注释须独占一行; 重复或未知字段被拒绝。env 覆盖继承的同名环境变量; 追加的 CLI 算法参数覆盖对应运行参数。

3.4 配置文件的填写样式

配置文件内容：自有数据示例（填写后保存，不在终端直接执行）

```
IMAGE=remote-sensing-framework:0.3.0-kmeans1
INPUT_DIR=/mnt/robot_disk/robot_data/待处理影像
OUTPUT_DIR=/mnt/robot_disk/robot_data/结果/task-001
WORK_DIR=/mnt/robot_disk/robot_data/工作/task-001
```

```
CONTAINER_NAME=rs-task-001
CPU_LIMIT=1
MEMORY_LIMIT=1g
STOP_TIMEOUT=10
RS_ALGORITHM=KMEANS
RS_INPUT=/data/input/scene.tif
RS_PARAMS={"clusters":3,"seed":0,"max_iter":100,"max_pixels":1000000}
RS_TASK_ID=task-001
```

需将示例目录和 scene.tif 替换为实际数据。当前输入波段使用原始数值，不自动标准化；正式数据的波段含义、缩放及聚类意义应另行确认。默认 env 不需要服务 URL、密码或访问令牌。

## 第四章 算法执行与参数参考

### 4.1 RUN-01 生成 KMEANS 验收样本

#### 用途

生成确定性的 90×90 双波段合成 GeoTIFF，用于封装验收。

#### 执行前提

已加载 IMAGE；仅在专用测试目录写入。已有同名样本可跳过生成。

#### 需填写内容

样本目录：示例固定 data/input/kmeans-example。正式影像无需执行此步骤。

#### 执行命令

```
mkdir -p data/input/kmeans-example
docker run --rm --network none \
  --user "$(id -u):$(id -g)" \
  -v "$PWD/data/input/kmeans-example:/fixture" \
  -v "$PWD/tests:/tests:ro" \
  --entrypoint python "$IMAGE" \
  /tests/create_kmeans_fixture.py /fixture
rc=$?
printf 'fixture_exit=%s\n' "$rc"
ls data/input/kmeans-example
```

| 参数                  | 用途                |
|---------------------|-------------------|
| --rm                | 退出后删除临时容器         |
| --network none      | 此容器禁网，生成过程使用已有依赖  |
| --user UID:GID      | 以宿主普通用户身份创建文件     |
| -v 宿主: 容器 [:ro]     | 挂载样本目录及只读测试脚本     |
| --entrypoint python | 运行样本生成脚本，替换算法入口   |
| /fixture [side]     | 输出目录及可选边长；缺省边长 90 |

#### 终端输出（格式示意）

```
fixture_exit=0
expected-labels.npy
kmeans-input.tif
```

示意中的任务名、时间、目录、哈希和耗时以实际运行值为准；该输出用于说明格式，不构成本次重新运行证据。

完成判定与异常处理

退出码 0，两个样本文件存在。生成脚本自身通常无 `stdout`；这里的状态文字来自 `printf` 和 `ls`。重新生成会覆盖指定测试样本，不得指向生产输入目录。

4.2 RUN-02 执行 env 配置任务

用途

运行指定算法并保存 `stdout`、JSON 日志及退出码。

执行前提

已完成 CFG-01 和输入准备；本终端保留 `ENV_FILE` 与 `TASK_ID`。

需填写内容

`ENV_FILE`：实际 `env` 文件；`TASK_ID`：用于宿主日志命名。若重新开终端，须手动恢复两者。

执行命令

```
mkdir -p logs
if python3 scripts/env-run.py "$ENV_FILE" \
  > "logs/$TASK_ID.stdout" 2> "logs/$TASK_ID.jsonl"; then
  rc=0
else
  rc=$?
fi
printf '%s\n' "$rc" > "logs/$TASK_ID.exit"
printf 'task=%s exit=%s\n' "$TASK_ID" "$rc"
tail -n 2 "logs/$TASK_ID.jsonl"
```

| 指令 / 参数                            | 说明                                                      |
|------------------------------------|---------------------------------------------------------|
| <code>env-run.py ENV_FILE</code>   | 按字面解析 <code>env</code> ，传给 <code>run.sh</code> ；不执行配置文本 |
| <code>&gt; stdout 文件</code>        | 保存标准输出；正常批处理时可能为空                                       |
| <code>2&gt; jsonl 文件</code>        | 保存标准错误通道上的逐行 JSON 日志                                    |
| <code>if ...; then ... else</code> | 无论成功失败都记录真实退出码                                          |
| <code>tail -n 2</code>             | 显示日志最后两行；长任务可能有更多日志                                     |

终端输出（格式示意）

```
task=kmeans-20260908-140000-12345 exit=0
{"level":"INFO","message":"algorithm_started","algorithm":"KMEANS", ...}
{"level":"INFO","message":"algorithm_completed","device":"cpu", ...}
```

示意中的任务名、时间、目录、哈希和耗时以实际运行值为准；该输出用于说明格式，不构成本次重新运行证据。

上面的省略号仅表示日志展示节选，不是完整合法JSON。完整日志保留在 jsonl 文件中；本机已保存的完成日志含 valid\_pixels=8099、invalid\_pixels=1、iterations=2，来源 reports/deployment/docker-integration/success.jsonl。

### 完成判定与异常处理

退出码为 0 且出现 algorithm\_completed。仍须执行 RUN-03 检查完成标记；单独看到启动日志不表示成功。退出非零时保留日志与输出目录，按故障章节定位。

## 4.3 RUN-03 检查结果和完成标记

### 用途

校验预期结果目录与任务状态。

### 执行前提

RUN-02 已结束。

### 需填写内容

RESULT\_DIR: 填写 env 中 OUTPUT\_DIR 下的 result 目录；下列值适用于 CFG-01 生成的配置。

### 执行命令

```
RESULT_DIR="$PROJECT_ROOT/outputs/$TASK_ID/result"
ls "$RESULT_DIR"
python3 - "$RESULT_DIR/success.json" <<'PYTHON'
import json, sys
from pathlib import Path
r = json.loads(Path(sys.argv[1]).read_text())
print("status=" + r["status"])
print("algorithm=" + r["algorithm"])
print("output=" + r["output"])
print("valid_pixels=" + str(r["valid_pixels"]))
assert r["status"] == "completed"
PYTHON
```

### 终端输出（格式示意）

```
clusters.tif
success.json
status=completed
algorithm=KMEANS
output=/data/output/result/clusters.tif
valid_pixels=8099
```

示意中的任务名、时间、目录、哈希和耗时以实际运行值为准；该输出用于说明格式，不构成本次重新运行证据。

### 完成判定与异常处理

任务退出码 0、状态 completed、期望 GeoTIFF 存在共同构成运行完成判定。output 字段是容器内路径；宿主对应 RESULT\_DIR。8099 仅适用于该合成样本，不是任意影像的固定结果。

## 4.4 KMEANS 参数表

| 参数         | 类型 / 默认      | 允许值及含义                     |
|------------|--------------|----------------------------|
| clusters   | 整数 / 3       | 2 至 255；聚类数；有效且不同像元数不得小于该值 |
| seed       | 整数 / 0       | 0 至 4294967295；随机种子        |
| max_iter   | 整数 / 100     | 1 至 10000；每次初始化的最大迭代次数     |
| max_pixels | 整数 / 1000000 | 1 至 10000000；允许的影像总像元数上限   |

布尔值不作为整数接收；未知参数报错。输入最多 32 波段，全部波段参与计算，NoData 和非有限像元被排除。固定 n\_init=10、单线程、Lloyd 算法；输出 uint16，0 为 NoData，1 至 K 为任意簇编号。允许的像元上限不保证对应内存足够；跨实现比较应考虑簇编号置换。

## 4.5 RUN-04 执行 MNDWI 示例

### 用途

计算绿光与 SWIR1 波段的 MNDWI，演示 CLI 参数覆盖。

### 执行前提

执行 ENV-01，存在镜像；目标输入为至少两个波段的专用测试 GeoTIFF。

### 需填写内容

本示例自动生成测试输入。自有影像应修改 INPUT\_DIR、-input 以及两个波段编号。

### 执行命令

```
mkdir -p data/input/mndwi-example logs
docker run --rm --network none \
  --user "$(id -u):$(id -g)" \
  -v "$PWD/data/input/mndwi-example:/fixture" \
  -v "$PWD/tests:/tests:ro" \
  --entrypoint python "$IMAGE" /tests/create_mndwi_fixture.py
TASK_ID="mndwi-$(date +%Y%m%d-%H%M%S)-$$"
if INPUT_DIR="$PWD/data/input/mndwi-example" \
  OUTPUT_DIR="$PWD/outputs/$TASK_ID" \
  WORK_DIR="$PWD/work/$TASK_ID" \
```

```
CONTAINER_NAME="rs-$TASK_ID" RS_TASK_ID="$TASK_ID" \
MEMORY_LIMIT=1g bash scripts/run.sh \
--algorithm MNDWI --input /data/input/mndwi-input.tif \
--params '{"green_band":1,"swir1_band":2,"output_name":"mndwi.tif"}' \
> "logs/$TASK_ID.stdout" 2> "logs/$TASK_ID.jsonl"; then
    rc=0
else
    rc=$?
fi
printf '%s\n' "$rc" > "logs/$TASK_ID.exit"
printf 'task=%s exit=%s\n' "$TASK_ID" "$rc"
ls "outputs/$TASK_ID/result"
```

终端输出（格式示意）

```
task=mndwi-20260908-141000-12345 exit=0
mndwi.tif
success.json
```

示意中的任务名、时间、目录、哈希和耗时以实际运行值为准；该输出用于说明格式，不构成本次重新运行证据。

完成判定与异常处理

退出码 0，结果文件与 success.json 存在。样本的数值校验由现有算法测试覆盖；真实影像另需确认反射率缩放与波段语义。

4.6 MNDWI 参数表

| 参数          | 类型 / 默认         | 填写规则                        |
|-------------|-----------------|-----------------------------|
| green_band  | 整数 / 1          | 从 1 开始，不能超过输入波段总数           |
| swir1_band  | 整数 / 2          | 从 1 开始，不能超过输入波段总数           |
| output_name | 字符串 / mndwi.tif | 普通文件名，后缀.tif 或.tiff；不接受目录路径 |

计算公式为 (G-S1)/(G+S1)。输出 float32，分母为零或输入无效的像元为 NaN；保留空间参考。当前按窗口处理，不提供网络服务。



## 4.7 算法命令行接口

| 参数                       | 用途                          | 对应配置          |
|--------------------------|-----------------------------|---------------|
| -algorithm               | 选择算法                        | RS_ALGORITHM  |
| -input                   | 容器内输入文件                     | RS_INPUT      |
| -output-dir              | 容器内输出目录；标准运行保持 /data/output | 由 run.sh 挂载决定 |
| -params                  | 算法 JSON 对象，外层用单引号包裹         | RS_PARAMS     |
| -task-id                 | 日志关联标识                      | RS_TASK_ID    |
| -help / -list / -version | 帮助、接入状态、版本；不执行算法            | 无             |
| -healthcheck             | 只检查 Python 和入口              | 不证明算法或挂载健康    |

### 执行命令：查看入口说明

```
docker run --rm --network none "$IMAGE" --help
docker run --rm --network none "$IMAGE" --version
```

### 终端输出（格式示意）

```
usage: ... [--algorithm ALGORITHM] [--input INPUT] ...
0.3.0
```

示意中的任务名、时间、目录、哈希和耗时以实际运行值为准；该输出用于说明格式，不构成本次重新运行证据。

帮助输出中的选项按镜像版本显示；-list 中 not\_implemented 的算法不可作为已交付能力使用。

# 第五章 运行监控、停止和重跑

## 5.1 CTL-01 查询状态与日志

### 用途

定位运行中的容器并查看进展。

### 执行前提

任务仍在运行；run.sh 使用 -rm，小任务结束后容器会自动删除。

### 需填写内容

CONTAINER: 填写 env 的 CONTAINER\_NAME；示例值必须改为实际运行名称。

### 执行命令

```
CONTAINER=rs-kmeans-001
docker ps --format 'table {{.Names}}\t{{.Status}}\t{{.Image}}'
docker logs --tail 20 "$CONTAINER"
docker stats --no-stream "$CONTAINER"
```

| 参数             | 用途                             |
|----------------|--------------------------------|
| -tail 20       | 只显示最近 20 条容器日志                 |
| --no-stream    | 获取一次资源快照后退出                    |
| docker logs -f | 可选持续跟踪；Ctrl+C 只结束查看            |
| CONTAINER      | Docker 任务名，不是 RS_TASK_ID 或镜像标签 |

### 终端输出（格式示意）

|               |               |                                        |                   |
|---------------|---------------|----------------------------------------|-------------------|
| NAMES         | STATUS        | IMAGE                                  |                   |
| rs-kmeans-001 | Up 12 seconds | remote-sensing-framework:0.3.0-kmeans1 |                   |
| CONTAINER ID  | NAME          | CPU %                                  | MEM USAGE / LIMIT |
| <id>          | rs-kmeans-001 | ...                                    | ...               |

示意中的任务名、时间、目录、哈希和耗时以实际运行值为准；该输出用于说明格式，不构成本次重新运行证据。

### 完成判定与异常处理

列表中的名字与目标一致。找不到容器时先查看宿主 logs 下的退出码；No such container 可能是 -rm 正常删除，而不是未运行。CPU、内存快照不是性能 SLA。

## 5.2 CTL-02 停止指定容器

### 用途

向指定任务请求正常终止，超时后由 Docker 强制终止。

### 执行前提

已确认容器名称，且没有将日志查看进程误认为算法进程。

### 需填写内容

CONTAINER：目标容器；GRACE\_SECONDS：非负整数，默认 10 秒。

### 执行命令

```
CONTAINER=rs-kmeans-001
GRACE_SECONDS=10
bash scripts/stop.sh "$CONTAINER" "$GRACE_SECONDS"
printf 'stop_command_exit=%s\n' "$?"
```

### 终端输出（格式示意）

```
rs-kmeans-001
stop_command_exit=0
```

示意中的任务名、时间、目录、哈希和耗时以实际运行值为准；该输出用于说明格式，不构成本次重新运行证据。

### 完成判定与异常处理

stop\_command\_exit=0 仅表示 Docker 停止命令成功。算法运行终端记录的退出码可能为 143，强杀时常见 137；两者不得混淆。原生计算库可能延迟处理信号，当前无断点续跑保证。

## 5.3 重跑规则

- 正常协作取消会尝试清理临时目录；OOM、强杀或断电可能留下.partial-\* 与.rs-lock。
- 重跑时建立新的任务标识、容器名和 OUTPUT\_DIR。保留旧配置、日志和结果目录，不在活动任务中清锁。
- 只有运行退出码与完整 success.json 均满足要求才标记完成。停止与发布结果发生竞争时记录两者，不将非零退出直接作为成功。

## 5.4 退出码参考

| 退出码             | 含义                      |
|-----------------|-------------------------|
| 0               | 操作成功；信息命令不代表算法执行成功      |
| 1               | 未处理错误                   |
| 2               | 参数、配置、输入内容或规模限制错误       |
| 3               | 算法未知或未接入                |
| 4               | 输入不存在/不可读，或算法输出 I/O 错误  |
| 5               | 结果已存在、目录占用，或 smoke 输出失败 |
| 130 / 143       | SIGINT / SIGTERM 协作中止   |
| 137             | 强杀或可能 OOM；需结合平台状态       |
| 125 / 126 / 127 | Docker 启动或入口执行错误        |

# 第六章 CPU 与 Docker GPU 验收

## 6.1 TEST-01 CPU 封装回归

### 用途

分别检查接口、容器运行约束和完整部署行为。

### 执行前提

设置 IMAGE；普通用户执行；完整验收会占用资源并创建诊断容器和输出。

### 需填写内容

IMAGE：待验收镜像；现有测试入口无额外必填 CLI 参数。

### 执行命令

```
bash scripts/test-local.sh
bash scripts/test-container.sh
python3 tests/verify_deployment.py
```

| 命令                   | 检查范围                    | 记录                       |
|----------------------|-------------------------|--------------------------|
| test-local.sh        | 接口与 env；算法库缺失可能跳过       | 终端 unittest 输出           |
| test-container.sh    | smoke、错误输入、用户和挂载权限      | outputs/container-test.* |
| verify_deployment.py | 算法、中文路径、env、权限、OOM、停止重跑 | outputs/deployment-*     |

### 终端输出（实测记录）

```
default UID verified
filesystem restrictions verified
PASS: 框架容器检查通过；不是遥感算法验收。记录目录：
↪ /mnt/robot_disk/remote_sensing_project/outputs/container-test.cyuHEB
```

依据：reports/deployment/container-checks.log。

### 终端输出（格式示意）

```
{"status": "PASS", "records": "/mnt/robot_disk/remote_sensing_project/outputs/deployment-< 本次目录>"}
```

示意中的任务名、时间、目录、哈希和耗时以实际运行值为准；该输出用于说明格式，不构成本次重新运行证据。

## 完成判定与异常处理

各命令均退出 0，相关检查输出 PASS；unittest 的跳过项需单独记录。完整部署测试中的 32 MiB OOM 是普通内存异常测试，不是 GPU 显存测试。

## 6.2 TEST-02 Docker GPU 计算

### 用途

验证容器内实际 CUDA 运算与 CPU 参考结果一致。

### 执行前提

宿主驱动、Toolkit 和 Docker GPU 运行时已配置；准备好本地镜像。

### 需填写内容

IMAGE：待测算法镜像；脚本缺省 0.3.0-kmeans1。本测试无输入影像和业务参数。

### 执行命令

```
nvidia-smi
nvidia-ctk --version
bash scripts/test-gpu.sh
```

### 终端输出（实测记录节选）

```
{
  "status": "PASS",
  "device": "NVIDIA GeForce GTX 1650",
  "operation": "float32 vector addition",
  "backend": "CUDA Driver API + PTX JIT",
  "cpu_fallback": false,
  "device_allocation_bytes": 12582996
}
```

依据：reports/deployment/gpu-followup/compute-result.json。仅列与判定有关的字段，省略内容不表示程序只输出这些字段。

### 终端输出（实测记录节选）

```
"runs": [
  {"seed": 0, "elements": 1048583, "max_absolute_error": 0.0},
  {"seed": 1, "elements": 1048583, "max_absolute_error": 0.0},
  {"seed": 2, "elements": 1048583, "max_absolute_error": 0.0}
]
```

依据：reports/deployment/gpu-followup/compute-result.json。仅列与判定有关的字段，省略内容不表示程序只输出这些字段。

## 完成判定与异常处理

status 为 PASS、cpu\_fallback 为 false，三轮 max\_absolute\_error 均为 0。nvidia-smi 单独成功不满足该判定。该测试分配约 12 MiB 显存，不证明真实模型的 CUDA 框架与显存需求已验收。

# 第七章 Kubernetes 操作与 GPU 调度

## 7.1 K8S-01 检查节点与导入算法镜像

### 用途

确认目标集群并向 kind 节点加载已有 Docker 镜像。

### 执行前提

执行 ENV-01；本机 rs-lab 已存在。

### 需填写内容

IMAGE：已验收镜像；集群名称固定 rs-lab，目标集群另行适配。

### 执行命令

```
kubectll config current-context
kubectll get nodes -o wide
export TMPDIR=/mnt/robot_disk/k8s/cache
kind load docker-image "$IMAGE" --name rs-lab
```

| 指令 / 参数                | 用途                       |
|------------------------|--------------------------|
| current-context        | 确认当前操作上下文，不切换集群          |
| -o wide                | 显示节点地址、运行时等扩展字段          |
| kind load docker-image | 将 Docker 镜像加载到指定 kind 节点 |
| --name rs-lab          | 指定本地集群                   |
| TMPDIR                 | 导入临时文件使用数据盘缓存            |

### 终端输出（格式示意）

```
kind-rs-lab
NAME                STATUS    ROLES          VERSION
rs-lab-control-plane Ready    control-plane  v1.36.4
Image ... loading...
```

示意中的任务名、时间、目录、哈希和耗时以实际运行值为准；该输出用于说明格式，不构成本次重新运行证据。

### 完成判定与异常处理

节点 Ready, 镜像导入命令退出 0; 镜像已存在时可能提示跳过。Docker 镜像库与节点 containerd 镜像库互相独立。



## 7.2 K8S-02 CPU Job 回归

### 用途

验证两个 CPU Job 并发运行及大样本 Job 的暂停取消。

### 执行前提

小样本已生成；IMAGE 已导入；本机 hostPath 映射已建立。

### 需填写内容

大样本目录固定 data/input/kmeans-large；边长 1000 对应 1000000 像元，不超过脚本参数缺省上限。

### 执行命令

```
kubectrl config set-context --current --namespace=default
mkdir -p data/input/kmeans-large
docker run --rm --network none \
  --user "$(id -u):$(id -g)" \
  -v "$PWD/data/input/kmeans-large:/fixture" \
  -v "$PWD/tests:/tests:ro" \
  --entrypoint python "$IMAGE" \
  /tests/create_kmeans_fixture.py /fixture 1000
python3 scripts/test-k8s.py
```

### 终端输出（格式示意）

```
Context "kind-rs-lab" modified.
{"status":"PASS","records":".../reports/deployment/k8s-< 时间戳>",
 "jobs":["rs-example-< 时间戳>-0","rs-example-< 时间戳>-1"],
 "cancelled":"rs-cancel-< 时间戳>"}
```

示意中的任务名、时间、目录、哈希和耗时以实际运行值为准；该输出用于说明格式，不构成本次重新运行证据。

### 完成判定与异常处理

status=PASS；小任务生成成功标记，取消任务无未清理结果。set-context 会修改本地 kubeconfig 的当前命名空间。结果位于 outputs/rs-example-\*, 证据位于 reports/deployment/k8s-\*。

## 7.3 K8S-03 GPU 完整调度测试

### 用途

验证 GPU 资源登记、实际计算、排队、释放、超额申请及超时。

### 执行前提

本集群总可分配 GPU 为 1，当前没有其他 GPU 任务；插件已就绪。

需填写内容

IMAGE 可由环境变量指定；KUBECONFIG / KUBECTL 可覆盖本机工具路径。不要并发启动多个本测试副本。

执行命令

```
kubectl get node rs-lab-control-plane \
  -o jsonpath='{.status.allocatable.nvidia\.com/gpu}{"\n"}'
kubectl -n kube-system get pods -l app=nvidia-device-plugin
python3 scripts/test-k8s-gpu.py
```

终端输出（实测记录）

```
{
  "status": "PASS",
  "allocatable_gpus": 1,
  "gpu_correctness": true,
  "exclusive_scheduling": true,
  "excess_request_unschedulable": true,
  "no_request_no_device": true,
  "job_deadline_exceeded_verified": true,
  "release_to_completion_seconds": 11.306236156000523,
  "completed_job": "queued-1788868434",
  "records": "/mnt/robot_disk/remote_sensing_project/reports/k8s-gpu/run-1788868434"
}
```

依据：reports/k8s-gpu/run-1788868434/summary.json。

| 结果字段                           | 完成含义                  |
|--------------------------------|-----------------------|
| gpu_correctness                | 实际 CUDA 计算正确          |
| exclusive_scheduling           | 保留配额时排队，释放后完成         |
| excess_request_unschedulable   | 请求两个 GPU 时无法调度        |
| no_request_no_device           | 未申请 GPU 的 Pod 看不到设备   |
| job_deadline_exceeded_verified | 实际触发 Job 超时           |
| release_to_completion_seconds  | 包含调度、启动、计算和轮询；不是纯调度延迟 |

完成判定与异常处理

status=PASS 且五个布尔检查均为 true。allocatable=1 表示容量，不表示实时空闲。测试先计算再空闲保留配额，不能证明运行中的长 CUDA 内核可即时无损停止。

## 7.4 K8S-04 手动提交 GPU Job

### 用途

单独提交可复现的 CUDA 计算任务。

### 执行前提

K8S-03 基础条件满足；已存在 configs/gpu/compute-job.json。

### 需填写内容

默认无需修改模板。若换镜像，应修改模板的 spec.template.spec.containers[0].image；仅 export IMAGE 不会改变该 JSON。

### 执行命令

```
kubectl create namespace rs-gpu-tests --dry-run=client -o yaml | \
  kubectl apply -f -
kubectl -n rs-gpu-tests create configmap rs-cuda-compute \
  --from-file=verify_cuda_compute.py=tests/verify_cuda_compute.py \
  --dry-run=client -o yaml | kubectl apply -f -
JOB=$(kubectl create -f configs/gpu/compute-job.json \
  -o jsonpath='{.metadata.name}')
printf 'JOB=%s\n' "$JOB"
kubectl -n rs-gpu-tests wait --for=condition=complete \
  "job/$JOB" --timeout=120s
kubectl -n rs-gpu-tests logs "job/$JOB"
```

| 参数 / 字段                  | 含义                    |
|--------------------------|-----------------------|
| -n rs-gpu-tests          | 目标命名空间                |
| --dry-run=client -o yaml | 生成对象描述，经 apply 创建或更新  |
| --from-file= 键 = 文件      | 把计算脚本加入 ConfigMap     |
| generateName             | 自动生成不冲突的 Job 名称       |
| --for=condition=complete | 等待 Job 完成；不是创建后立即返回成功 |
| --timeout=120s           | 本地 wait 命令最长等待时间      |

### 终端输出（格式示意）

```
namespace/rs-gpu-tests configured
configmap/rs-cuda-compute created
JOB=rs-cuda-check-9d229
job.batch/rs-cuda-check-9d229 condition met
{"status":"PASS","device":"NVIDIA GeForce GTX 1650", ...}
```

示意中的任务名、时间、目录、哈希和耗时以实际运行值为准；该输出用于说明格式，不构成本次重新运行证据。

创建/配置提示依赖对象是否已存在。实际独立运算证据保存于 `reports/k8s-gpu/manual-job.log`。此处 Job 名以实际生成值为准，JSON 省略号仅用于展示。

### 7.5 手动 GPU Job 模板参数

| 字段                            | 当前值                      | 修改规则                   |
|-------------------------------|--------------------------|------------------------|
| image / imagePullPolicy       | 0.3.0-kmeans1 /<br>Never | 修改镜像后先导入节点；Never 不在线拉取 |
| requests.nvidia.com/gpu       | 1                        | 本机只有 1；多卡目标另行设计        |
| limits.nvidia.com/gpu         | 1                        | 与申请数量一致                |
| requests.cpu / memory         | 100m / 128Mi             | 调度请求                   |
| limits.cpu / memory           | 1 / 512Mi                | 容器 CPU / RAM 上限，不是显存   |
| activeDeadlineSeconds         | 120                      | Job 生命周期时限             |
| terminationGracePeriodSeconds | 10                       | Pod 正常终止宽限             |
| backoffLimit                  | 0                        | 失败不自动重试                |
| runAsUser / runAsGroup        | 1000 / 1000              | 模板固定值；不是动态读取当前 UID     |

普通计算 Pod 使用默认 runc 与 CDI 注入，不需要 privileged、显卡 hostPath 或 nvidia-bootstrap RuntimeClass。宿主桌面和外部 Docker 程序仍可竞争显存；Kubernetes 单卡配额不构成整卡性能或显存隔离。

### 7.6 K8S-05 查看、暂停与保留 Job 记录

#### 用途

保存任务日志并暂停整个 Job，避免控制器重建被单独删除的 Pod。

#### 执行前提

已知实际 Job 名称及其命名空间。

#### 需填写内容

NS：命名空间；JOB：K8S-04 返回的名称，或查询所得 CPU Job 名称。

#### 执行命令

```
NS=rs-gpu-tests
# 本终端沿用 K8S-04 的 JOB；新终端须先填写 JOB= 实际名称。
kubectl -n "$NS" get jobs,pods -o wide
kubectl -n "$NS" describe "job/$JOB"
mkdir -p logs
kubectl -n "$NS" logs "job/$JOB" > "logs/$JOB.log"
```

```
kubecttl -n "$NS" patch "job/$JOB" --type=merge \
  -p '{"spec":{"suspend":true}}'
kubecttl -n "$NS" get "job/$JOB" \
  -o jsonpath='{.spec.suspend}{"\n"}'
```

### 终端输出（格式示意）

```
job.batch/rs-cuda-check-9d229 patched
true
```

示意中的任务名、时间、目录、哈希和耗时以实际运行值为准；该输出用于说明格式，不构成本次重新运行证据。

### 完成判定与异常处理

suspend=true 表示暂停设置生效，活动 Pod 的删除需要等待终止过程；已完成 Job 无运行进程可停止。恢复会重建 Pod，不等于算法断点续算。归档后只清理明确的测试对象。

# 第八章 本地集群维护

## 8.1 MAINT-01 创建缺失的 kind 集群

### 用途

在指定数据盘上建立 rs-lab。

### 执行前提

仅在 rs-lab 不存在时使用；kind、kubectl 已安装，Docker 数据目录位于指定数据盘。

### 需填写内容

当前脚本固定 /mnt/robot\_disk/k8s 与 rs-lab；不提供通用安装参数。

### 执行命令

```
bash scripts/create-kind.sh
```

### 终端输出（格式示意）

```
Creating cluster "rs-lab" ...  
Set kubectl context to "kind-rs-lab"
```

示意中的任务名、时间、目录、哈希和耗时以实际运行值为准；该输出用于说明格式，不构成本次重新运行证据。

### 完成判定与异常处理

以 kubectl get nodes 显示 Ready 为准。集群已存在时脚本会保留配置并退出 2，提示 rs-lab 已存在；不得将该情形解释为需要删除现有集群。

## 8.2 MAINT-02 复装本机 GPU 接入

### 用途

复制当前宿主驱动到 kind 节点，配置初始化运行时并安装设备插件。

### 执行前提

维护窗口内执行；没有活动或 Pending GPU Pod。宿主驱动和 Toolkit 已完成配置。

### 需填写内容

脚本固定本机节点和官方插件摘要；驱动升级、其他操作系统或生产集群应制定独立方案。

执行命令

```
python3 scripts/setup-kind-gpu.py
bash scripts/install-kind-device-plugin.sh
python3 scripts/test-k8s-gpu.py
```

终端输出（实测记录）

PASS: kind driver root, CDI and bootstrap runtime prepared

依据：reports/k8s-gpu/setup-replay.log。

终端输出（实测记录节选）

daemon set "nvidia-device-plugin" successfully rolled out  
PASS: nvidia.com/gpu=1

依据：reports/k8s-gpu/install-replay.log。仅列与判定有关的字段，省略内容不表示程序只输出这些字段。

完成判定与异常处理

准备、安装和完整复测均通过。准备脚本会删除自己的插件并重启节点内 containerd；不是日常运行前置步骤。首次缺少插件镜像需联网。驱动副本不是自动升级机制，不能直接照搬到生产节点。

8.3 组件与路径

| 对象                        | 作用                          |
|---------------------------|-----------------------------|
| /opt/nvidia-driver        | 节点内驱动副本                     |
| /etc/cdi/nvidia-kind.json | 插件初始化用 CDI 描述               |
| /run/cdi/                 | 设备插件生成的任务 CDI 描述            |
| nvidia-bootstrap          | 只供设备插件初始化使用的 RuntimeClass   |
| device-plugin DaemonSet   | 向 kubelet 登记 nvidia.com/gpu |

install-docker.sh 和 install-docker-retry.sh 是本机首次安装历史脚本，包含用户和磁盘身份限制。当前 Docker 已存在，不应作为每次部署或目标机器的通用安装步骤执行。

# 第九章 构建、安全扫描与离线交付

## 9.1 REL-01 构建新的镜像版本

### 用途

固定基础镜像摘要和构建编号，生成独立算法镜像。

### 执行前提

构建网络可用，源码与依赖已审查；记录工作区修改。

### 需填写内容

PLATFORM: 本手册为 linux/amd64; BUILD\_ID: 唯一编号; PYTHON\_BASE: 已核验摘要。

### 执行命令

```
export PYTHON_BASE=python:3.13-slim-trixie@sha256:9d2e5553305c7c7b0097999bb17187c69b921j
↪ ccd6bc9d40e4bb5ebe652c00285
export PLATFORM=linux/amd64
export BUILD_ID="$(git rev-parse --short HEAD)-$(date +%Y%m%d%H%M%S)"
bash scripts/build.sh
export IMAGE="remote-sensing-framework:$(cat VERSION)-$BUILD_ID"
printf 'IMAGE=%s\n' "$IMAGE"
```

| 变量               | 要求                             |
|------------------|--------------------------------|
| PYTHON_BASE      | 必须包含 @sha256: 后接 64 位十六进制摘要    |
| BUILD_ID         | 字母或数字开头，仅允许字母、数字、下划线、点、连字符     |
| PLATFORM         | 明确指定目标架构，不将本机架构视为甲方要求          |
| IMAGE_REPOSITORY | 可选；缺省 remote-sensing-framework |

### 终端输出（格式示意）

```
... 构建过程输出 ...
remote-sensing-framework:0.3.0-e59ce62-20260908150000
IMAGE=remote-sensing-framework:0.3.0-e59ce62-20260908150000
```

示意中的任务名、时间、目录、哈希和耗时以实际运行值为准；该输出用于说明格式，不构成本次重新运行证据。

### 完成判定与异常处理



构建命令退出 0，并对新 IMAGE 执行 TEST-01。当前构建上下文包含工作区文件，提交号本身不能证明未提交源码未参与构建。该基础摘要是当前项目基线，不能视为永久安全版本。

## 9.2 REL-02 扫描镜像漏洞

### 用途

生成 HIGH/CRITICAL 漏洞 JSON，并用退出码实施门禁。

### 执行前提

Trivy 已安装；在线扫描需要数据库更新网络。

### 需填写内容

IMAGE：待扫描镜像；TRIVY：实际可执行文件；SCAN\_DIR：本次独立报告目录。

### 执行命令

```
export TRIVY=/mnt/robot_disk/tools/trivy-0.74.0/trivy
export SCAN_DIR="$PWD/reports/security/scan-$(date +%Y%m%d-%H%M%S)"
if bash scripts/scan-image.sh; then
    scan_rc=0
else
    scan_rc=$?
fi
printf 'scan_exit=%s\n' "$scan_rc"
ls "$SCAN_DIR"
```

### 终端输出（格式示意）

```
scan_exit=2
scan.json
tool-version.txt
```

示意中的任务名、时间、目录、哈希和耗时以实际运行值为准；该输出用于说明格式，不构成本次重新运行证据。

### 完成判定与异常处理

0 表示本次所选级别未命中；2 表示高危门禁命中；其他非零需检查工具或网络故障。文件存在并不代表扫描通过。复用旧数据库时必须保留数据库时间。

### 9.3 已记录安全状态

| 组件                 | HIGH / CRITICAL | 结论       |
|--------------------|-----------------|----------|
| 算法镜像 0.3.0-kmeans1 | 47 / 3          | 尚未达到安全门禁 |
| 设备插件 v0.20.0       | 2 / 0           | 尚未达到安全门禁 |

算法镜像为 50 条软件包-漏洞记录、18 个不同编号。插件两条均为 CVE-2026-14456，扫描器列出了修复版本。以上来自同日历史扫描，本次文档修订未重新扫描或修复。

### 9.4 REL-03 导出单个镜像

#### 用途

生成可 docker load 的压缩镜像及 SHA256 文件。

#### 执行前提

IMAGE 已完成相应验收；导出目标有足够空间。

#### 需填写内容

EXPORT\_FILE：新的导出文件路径，现有文件会被拒绝覆盖。

#### 执行命令

```
export EXPORT_FILE="/mnt/robot_disk/deliveries/image-$(date +%Y%m%d-%H%M%S).tar.gz"
bash scripts/export.sh
sha256sum -c "$EXPORT_FILE.sha256"
```

#### 终端输出（格式示意）

```
/mnt/robot_disk/deliveries/image-20260908-150500.tar.gz: OK
```

示意中的任务名、时间、目录、哈希和耗时以实际运行值为准；该输出用于说明格式，不构成本次重新运行证据。

#### 完成判定与异常处理

export.sh 通常无成功提示；以退出 0、两个文件存在和校验 OK 为准。该产物仅含镜像，不包含配置、输入数据、GPU 驱动或运维工具。

### 9.5 REL-04 加载已有 CPU 离线交付包

#### 用途

验证外层及内部校验文件后离线加载镜像。

执行前提

目标为 Linux amd64，已有 Docker、Bash、Python 3、gzip 和 sha256sum。

需填写内容

BUNDLE\_DIR：压缩包所在目录；PACKAGE\_ROOT：解包后 SHA256SUMS 所在目录。

执行命令

```
BUNDLE_DIR=/mnt/robot_disk/deliveries
cd "$BUNDLE_DIR"
sha256sum -c rs-0.3.0-kmeans1-bundle.tar.gz.sha256
EXTRACT_DIR="/mnt/robot_disk/tmp/rs-load-$(date +%Y%m%d-%H%M%S)"
mkdir -p "$EXTRACT_DIR"
tar -xzf rs-0.3.0-kmeans1-bundle.tar.gz -C "$EXTRACT_DIR"
find "$EXTRACT_DIR" -name SHA256SUMS -print
```

终端输出（格式示意）

```
rs-0.3.0-kmeans1-bundle.tar.gz: OK
/mnt/robot_disk/tmp/rs-load-20260908-151000/rs-0.3.0-kmeans1/SHA256SUMS
```

示意中的任务名、时间、目录、哈希和耗时以实际运行值为准；该输出用于说明格式，不构成本次重新运行证据。

将 find 返回路径末尾的 /SHA256SUMS 去掉，填写为 PACKAGE\_ROOT。以下示例目录须按实际结果替换，不假定压缩包内部目录固定。

执行命令

```
PACKAGE_ROOT=/mnt/robot_disk/tmp/rs-load-20260908-151000/rs-0.3.0-kmeans1
cd "$PACKAGE_ROOT"
sha256sum -c SHA256SUMS
gzip -dc image.tar.gz | docker load
test -e .env || cp .env.example .env
```

| 参数                 | 含义                     |
|--------------------|------------------------|
| sha256sum -c       | 按清单校验文件；必须全部 OK        |
| tar -xzf ... -C    | 解压 gzip tar 到指定新目录     |
| gzip -dc           | 解压到 stdout，不删除原压缩包     |
| docker load        | 从标准输入加载镜像              |
| test -e .env    cp | 仅在.env 不存在时复制，避免覆盖用户配置 |

终端输出（格式示意）

```
image.tar.gz: OK
... 其他交付文件: OK
Loaded image: remote-sensing-framework:0.3.0-kmeans1
```

示意中的任务名、时间、目录、哈希和耗时以实际运行值为准；该输出用于说明格式，不构成本次重新运行证据。

### 完成判定与异常处理

全部内部校验通过且镜像成功加载后，按配置及执行章节运行。若在解包目录工作，PROJECT\_ROOT 应设为该目录。离线 CPU 包不包含完整 GPU/K8s 基础设施；旧包也不自动包含本修订手册。

## 9.6 REL-05 使用现有打包入口

### 用途

生成当前脚本定义的交付目录，包含镜像、样本、源码及指定文档。

### 执行前提

IMAGE 已验证；输出目录尚不存在。正式完整交付前需补齐文档清单。

### 需填写内容

PACKAGE\_DIR：新目录绝对路径。

### 执行命令

```
cd "$PROJECT_ROOT"
export PACKAGE_DIR="/mnt/robot_disk/deliveries/rs-package-$(date +%Y%m%d-%H%M%S)"
bash scripts/package.sh
```

### 终端输出（格式示意）

```
交付目录: /mnt/robot_disk/deliveries/rs-package-20260908-152000
```

示意中的任务名、时间、目录、哈希和耗时以实际运行值为准；该输出用于说明格式，不构成本次重新运行证据。

### 完成判定与异常处理

此命令生成目录，不自动生成外层 bundle.tar.gz。当前 package.sh 固定文档清单未包含本修订手册和新 GPU 总报告；不得将输出直接标为文档齐全的新交付版。增补文件后必须重建并校验 SHA256SUMS。本次未修改打包脚本或旧包。

# 第十章 源码推送与文件发布

## 10.1 GIT-01 推送已提交开发分支

### 用途

将已有本地提交上传到 origin 的开发分支。

### 执行前提

仓库 SSH 凭据可用；所需文件已经明确提交。

### 需填写内容

分支为 codex/kmeans-deployment；如分支变更，应先核对再调整。

### 执行命令

```
cd /mnt/robot_disk/remote_sensing_project
git status --short
git branch --show-current
git log -3 --oneline
git push -u origin codex/kmeans-deployment
```

| 参数                    | 含义                    |
|-----------------------|-----------------------|
| status --short        | 检查尚未提交的修改             |
| branch --show-current | 确认当前分支                |
| log -3 --oneline      | 确认最近三个提交              |
| push -u origin 分支     | 推送该分支并设置上游跟踪；不合并 main |

### 终端输出（格式示意）

```
To github.com:huam54925-cpu/remote-sensing-algorithms.git
* [new branch]      codex/kmeans-deployment -> codex/kmeans-deployment
branch ... set up to track ...
```

示意中的任务名、时间、目录、哈希和耗时以实际运行值为准；该输出用于说明格式，不构成本次重新运行证据。

### 完成判定与异常处理

实际输出也可能是增量更新或 Everything up-to-date。git push 不上传未提交手册，也不上传 Docker 镜像。新手册需要单独审阅、提交后才可推送；本次仅生成文档，不自动提交或发布。

# 第十一章 故障定位与交付验收

## 11.1 故障对照表

| 可见现象                        | 检查项目                        | 操作结论              |
|-----------------------------|-----------------------------|-------------------|
| 输入或输出文件错误                   | 目录权限、RS_INPUT、只读挂载          | 纠正路径/权限后采用新任务目录   |
| result 已存在                  | OUTPUT_DIR/result           | 换新输出目录，不覆盖旧成果     |
| 退出 137                      | OOMKilled、终止记录、日志           | 区分 OOM 与强杀        |
| GPU 不可见                     | 宿主 nvidia-smi、Docker GPU 测试 | 逐层定位，不直接修改算法      |
| Insufficient nvidia.com/gpu | 申请量与当前配额占用                  | 等待或减少请求；本机总量为 1   |
| ErrImageNeverPull           | 节点镜像与模板 image               | 导入完全一致的镜像         |
| CDI mount 错误                | Pod 事件、节点驱动文件               | 维护窗口检查 CDI 引用     |
| DeadlineExceeded            | Job 时限、任务进展                 | 确认是正常超时、资源不足或程序卡住 |

## 11.2 DIAG-01 保存集群故障记录

### 用途

收集排错所需状态，保留可复现信息。

### 执行前提

已设置 KUBECONFIG；目标命名空间存在。

### 需填写内容

NS：实际命名空间；RECORD 自动生成。单任务应另保存其日志、env 和退出码，外发前去除凭据。

### 执行命令

```
NS=rs-gpu-tests
RECORD="logs/diagnosis-$(date +%Y%m%d-%H%M%S) - $$"
mkdir -p "$RECORD"
date -Is > "$RECORD/time.txt"
docker version > "$RECORD/docker-version.txt" 2>&1
findmnt -T "$PROJECT_ROOT" > "$RECORD/mount.txt"
nvidia-smi > "$RECORD/gpu.txt" 2>&1
kubectl get nodes -o wide > "$RECORD/nodes.txt" 2>&1
```

```
kubecttl -n "$NS" get jobs,pods -o wide \
> "$RECORD/tasks.txt" 2>&1
kubecttl -n "$NS" get events \
--sort-by=.metadata.creationTimestamp \
> "$RECORD/events.txt" 2>&1
printf 'RECORD=%s\n' "$RECORD"
```

终端输出（格式示意）

```
RECORD=logs/diagnosis-20260908-153000-12345
```

示意中的任务名、时间、目录、哈希和耗时以实际运行值为准；该输出用于说明格式，不构成本次重新运行证据。

完成判定与异常处理

目录创建成功不等于所有诊断命令成功；逐个查看文件，命令错误也可能被重定向保存。不要公开 kubeconfig、私钥或完整环境变量。容器 `-rm` 后的日志应依靠启动时保存的宿主日志。

11.3 交付完成条件

- 配置：必填字段明确，样例能运行；宿主与容器路径对应。
- 结果：退出码、成功标记和期望文件一致；真实数据按对方基准比较。
- 生命周期：启动、日志、停止、异常和重跑行为均有验证记录。
- 资源：CPU/RAM/GPU 需求经过目标负载验证；GPU 诊断不替代真实框架测试。
- 部署：干净机器或目标服务器完成独立离线演练。
- 安全：高危门禁满足要求，组件、许可证、权重和 Secrets 处理明确。
- 交付物：镜像、env、命令手册、数据及校验清单版本一致。

11.4 待真实代码到达后填写的信息

| 类别    | 应提供或确认的内容                  |
|-------|----------------------------|
| 入口与依赖 | 源程序入口、依赖锁定、框架/CUDA 版本、构建步骤 |
| 输入    | 标准数据、波段、缩放、NoData、CRS、尺寸限制 |
| 输出    | 标准结果、文件格式、误差容限、成功判定        |
| 资源与停止 | 峰值内存/显存、运行时长、子进程、取消与检查点    |
| 联网与服务 | 权重下载、授权、外部接口、是否需常驻 API     |
| 目标平台  | 架构、运行时、K8s、存储、镜像仓库和权限      |

当前尚未完成的重点包括漏洞修复、异机验收、磁盘满、大规模输入、并发写入、长 GPU 内核取消、子进程清理、日志留存策略、Kubernetes 网络策略与生产存储。该清单不将尚未实现的能力作为交付承诺。



## 第十二章 附录：脚本、证据与文档维护

### 12.1 脚本与实际作用

| 脚本                            | 实际作用                               |
|-------------------------------|------------------------------------|
| env-run.py → run.sh           | 解析配置，组装挂载和资源限制，exec Docker         |
| stop.sh                       | 调用 docker stop，传入任务名与宽限秒数          |
| test-gpu.sh                   | 调用已有 verify_cuda_compute.py，执行诊断内核 |
| test-k8s-gpu.py               | 创建测试资源、检查调度与计算、保存证据                |
| setup-kind-gpu.py             | 准备 kind 驱动副本、CDI 与专用运行时            |
| install-kind-device-plugin.sh | 导入固定插件镜像、部署并检查资源登记                 |
| export.sh / package.sh        | 镜像导出 / 交付目录制作                      |
| inspect-algorithm.py          | 只读搜索依赖及调用线索，不执行收到的代码               |

### 12.2 日志和结果字段

| 字段                            | 解释                                                       |
|-------------------------------|----------------------------------------------------------|
| timestamp / level / module    | 日志时间、级别、模块                                               |
| task_id / algorithm           | 关联任务和算法                                                  |
| message                       | algorithm_started、algorithm_completed、task_cancelled 等事件 |
| elapsed_seconds               | 进程内部记录耗时，不等于完整平台延迟                                       |
| process_peak_rss_kib          | 进程 RSS 峰值，不含完整 cgroup 或 GPU 显存                           |
| valid_pixels / invalid_pixels | 有效与无效像素数                                                 |
| status=completed              | success.json 的执行完成状态                                     |

### 12.3 实测证据位置

- 总体状态：docs/readiness-report-2026-09-08.md。
- CPU 演练及退出码：reports/deployment/docker-integration/。
- Docker GPU：reports/deployment/gpu-followup/compute-result.json。
- Kubernetes GPU：reports/k8s-gpu/run-1788868434/summary.json。
- 手动 GPU Job：reports/k8s-gpu/manual-job.log。

- 插件扫描: reports/k8s-gpu/plugin-security.json。

## 12.4 文档源文件与编译

本版提供 Markdown 文本版和 LaTeX 源码。Markdown 用于复制命令; LaTeX 用于排版 PDF。修订时应同步维护内容, 不将终端输出示例复制为执行指令。

### 执行命令

```
cd /mnt/robot_disk/remote_sensing_project
bash docs/manual/build.sh
sha256sum -c output/pdf/remote-sensing-operations-manual.pdf.sha256
```

### 终端输出 (格式示意)

```
PDF: /mnt/robot_disk/remote_sensing_project/output/pdf/remote-sensing-operations-manual.pdf
output/pdf/remote-sensing-operations-manual.pdf: OK
```

示意中的任务名、时间、目录、哈希和耗时以实际运行值为准; 该输出用于说明格式, 不构成本次重新运行证据。

编译使用 XeLaTeX, 不执行文档内命令。本版按要求不逐页视觉检查, 采用编译、字体缺字、文本、目录、命令块语法检查。代码块形式可复用不代表本次已执行其中每项维护操作。